

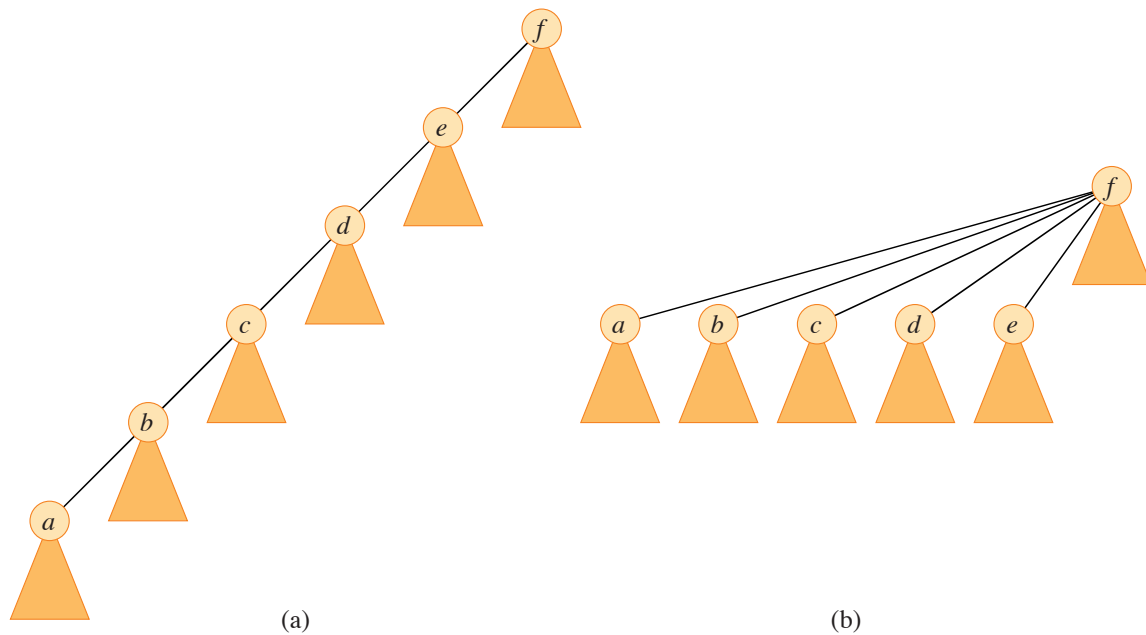


Figure 19.5 Path compression during the operation FIND-SET. Arrows and self-loops at roots are omitted. **(a)** A tree representing a set prior to executing FIND-SET(a). Triangles represent subtrees whose roots are the nodes shown. Each node has a pointer to its parent. **(b)** The same set after executing FIND-SET(a). Each node on the find path now points directly to the root.

corresponding tree has an initial rank of 0. Each FIND-SET operation leaves all ranks unchanged. The UNION operation has two cases, depending on whether the roots of the trees have equal rank. If the roots have unequal ranks, make the root with higher rank the parent of the root with lower rank, but don't change the ranks themselves. If the roots have equal ranks, arbitrarily choose one of the roots as the parent and increment its rank.

Let's put this method into pseudocode, appearing on the next page. The parent of node x is denoted by $x.p$. The LINK procedure, a subroutine called by UNION, takes pointers to two roots as inputs. The FIND-SET procedure with path compression, implemented recursively, turns out to be quite simple.

The FIND-SET procedure is a *two-pass method*: as it recurses, it makes one pass up the find path to find the root, and as the recursion unwinds, it makes a second pass back down the find path to update each node to point directly to the root. Each call of FIND-SET(x) returns $x.p$ in line 3. If x is the root, then FIND-SET skips line 2 and just returns $x.p$, which is x . In this case the recursion bottoms out. Otherwise, line 2 executes, and the recursive call with parameter $x.p$ returns

```

MAKE-SET( $x$ )
1   $x.p = x$ 
2   $x.rank = 0$ 

UNION( $x, y$ )
1  LINK(FIND-SET( $x$ ), FIND-SET( $y$ ))

LINK( $x, y$ )
1  if  $x.rank > y.rank$ 
2       $y.p = x$ 
3  else  $x.p = y$ 
4      if  $x.rank == y.rank$ 
5           $y.rank = y.rank + 1$ 

FIND-SET( $x$ )
1  if  $x \neq x.p$                 // not the root?
2       $x.p = \text{FIND-SET}(x.p)$     // the root becomes the parent
3  return  $x.p$                   // return the root

```

a pointer to the root. Line 2 updates node x to point directly to the root, and line 3 returns this pointer.

Effect of the heuristics on the running time

Separately, either union by rank or path compression improves the running time of the operations on disjoint-set forests, and combining the two heuristics yields an even greater improvement. Alone, union by rank yields a running time of $O(m \lg n)$ for a sequence of m operations, n of which are MAKE-SET (see Exercise 19.4-4), and this bound is tight (see Exercise 19.3-3). Although we won't prove it here, for a sequence of n MAKE-SET operations (and hence at most $n - 1$ UNION operations) and f FIND-SET operations, the worst-case running time using only the path-compression heuristic is $\Theta(n + f \cdot (1 + \log_{2+f/n} n))$.

Combining union by rank and path compression gives a worst-case running time of $O(m\alpha(n))$, where $\alpha(n)$ is a *very* slowly growing function, defined in Section 19.4. In any conceivable application of a disjoint-set data structure, $\alpha(n) \leq 4$, and thus, its running time is as good as linear in m for all practical purposes. Mathematically speaking, however, it is superlinear. Section 19.4 proves this $O(m\alpha(n))$ upper bound.

Exercises**19.3-1**

Redo Exercise 19.2-2 using a disjoint-set forest with union by rank and path compression. Show the resulting forest with each node including its x_i and rank.

19.3-2

Write a nonrecursive version of FIND-SET with path compression.

19.3-3

Give a sequence of m MAKE-SET, UNION, and FIND-SET operations, n of which are MAKE-SET operations, that takes $\Omega(m \lg n)$ time when using only union by rank and not path compression.

19.3-4

Consider the operation PRINT-SET(x), which is given a node x and prints all the members of x 's set, in any order. Show how to add just a single attribute to each node in a disjoint-set forest so that PRINT-SET(x) takes time linear in the number of members of x 's set and the asymptotic running times of the other operations are unchanged. Assume that you can print each member of the set in $O(1)$ time.

★ 19.3-5

Show that any sequence of m MAKE-SET, FIND-SET, and LINK operations, where all the LINK operations appear before any of the FIND-SET operations, takes only $O(m)$ time when using both path compression and union by rank. You may assume that the arguments to LINK are roots within the disjoint-set forest. What happens in the same situation when using only path compression and not union by rank?

★ 19.4 Analysis of union by rank with path compression

As noted in Section 19.3, the combined union-by-rank and path-compression heuristic runs in $O(m \alpha(n))$ time for m disjoint-set operations on n elements. In this section, we'll explore the function α to see just how slowly it grows. Then we'll analyze the running time using the potential method of amortized analysis.

A very quickly growing function and its very slowly growing inverse

For integers $j, k \geq 0$, we define the function $A_k(j)$ as

$$A_k(j) = \begin{cases} j + 1 & \text{if } k = 0, \\ A_{k-1}^{(j+1)}(j) & \text{if } k \geq 1, \end{cases} \quad (19.1)$$

where the expression $A_{k-1}^{(j+1)}(j)$ uses the functional-iteration notation defined in equation (3.30) on page 68. Specifically, equation (3.30) gives $A_{k-1}^{(0)}(j) = j$ and $A_{k-1}^{(i)}(j) = A_{k-1}(A_{k-1}^{(i-1)}(j))$ for $i \geq 1$. We call the parameter k the *level* of the function A .

The function $A_k(j)$ strictly increases with both j and k . To see just how quickly this function grows, we first obtain closed-form expressions for $A_1(j)$ and $A_2(j)$.

Lemma 19.2

For any integer $j \geq 1$, we have $A_1(j) = 2j + 1$.

Proof We first use induction on i to show that $A_0^{(i)}(j) = j + i$. For the base case, $A_0^{(0)}(j) = j = j + 0$. For the inductive step, assume that $A_0^{(i-1)}(j) = j + (i - 1)$. Then $A_0^{(i)}(j) = A_0(A_0^{(i-1)}(j)) = (j + (i - 1)) + 1 = j + i$. Finally, we note that $A_1(j) = A_0^{(j+1)}(j) = j + (j + 1) = 2j + 1$. ■

Lemma 19.3

For any integer $j \geq 1$, we have $A_2(j) = 2^{j+1}(j + 1) - 1$.

Proof We first use induction on i to show that $A_1^{(i)}(j) = 2^i(j + 1) - 1$. For the base case, we have $A_1^{(0)}(j) = j = 2^0(j + 1) - 1$. For the inductive step, assume that $A_1^{(i-1)}(j) = 2^{i-1}(j + 1) - 1$. Then $A_1^{(i)}(j) = A_1(A_1^{(i-1)}(j)) = A_1(2^{i-1}(j + 1) - 1) = 2 \cdot (2^{i-1}(j + 1) - 1) + 1 = 2^i(j + 1) - 2 + 1 = 2^i(j + 1) - 1$. Finally, we note that $A_2(j) = A_1^{(j+1)}(j) = 2^{j+1}(j + 1) - 1$. ■

Now we can see how quickly $A_k(j)$ grows by simply examining $A_k(1)$ for levels $k = 0, 1, 2, 3, 4$. From the definition of $A_0(j)$ and the above lemmas, we have $A_0(1) = 1 + 1 = 2$, $A_1(1) = 2 \cdot 1 + 1 = 3$, and $A_2(1) = 2^{1+1} \cdot (1 + 1) - 1 = 7$. We also have

$$\begin{aligned} A_3(1) &= A_2^{(2)}(1) \\ &= A_2(A_2(1)) \\ &= A_2(7) \\ &= 2^8 \cdot 8 - 1 \\ &= 2^{11} - 1 \\ &= 2047 \end{aligned}$$

and

$$\begin{aligned}
 A_4(1) &= A_3^{(2)}(1) \\
 &= A_3(A_3(1)) \\
 &= A_3(2047) \\
 &= A_2^{(2048)}(2047) \\
 &\gg A_2(2047) \\
 &= 2^{2048} \cdot 2048 - 1 \\
 &= 2^{2059} - 1 \\
 &> 2^{2056} \\
 &= (2^4)^{514} \\
 &= 16^{514} \\
 &\gg 10^{80},
 \end{aligned}$$

which is the estimated number of atoms in the observable universe. (The symbol “ $\gg$ ” denotes the “much-greater-than” relation.)

We define the inverse of the function $A_k(n)$, for integer $n \geq 0$, by

$$\alpha(n) = \min \{k : A_k(1) \geq n\}. \quad (19.2)$$

In words, $\alpha(n)$ is the lowest level k for which $A_k(1)$ is at least n . From the above values of $A_k(1)$, we see that

$$\alpha(n) = \begin{cases} 0 & \text{for } 0 \leq n \leq 2, \\ 1 & \text{for } n = 3, \\ 2 & \text{for } 4 \leq n \leq 7, \\ 3 & \text{for } 8 \leq n \leq 2047, \\ 4 & \text{for } 2048 \leq n \leq A_4(1). \end{cases}$$

It is only for values of n so large that the term “astronomical” understates them (greater than $A_4(1)$, a huge number) that $\alpha(n) > 4$, and so $\alpha(n) \leq 4$ for all practical purposes.

Properties of ranks

In the remainder of this section, we prove an $O(m\alpha(n))$ bound on the running time of the disjoint-set operations with union by rank and path compression. In order to prove this bound, we first prove some simple properties of ranks.

Lemma 19.4

For all nodes x , we have $x.\text{rank} \leq x.p.\text{rank}$, with strict inequality if $x \neq x.p$ (x is not a root). The value of $x.\text{rank}$ is initially 0, increases through time until $x \neq x.p$,

and from then on, $x.rank$ does not change. The value of $x.p.rank$ monotonically increases over time.

Proof The proof is a straightforward induction on the number of operations, using the implementations of MAKE-SET, UNION, and FIND-SET that appear on page 530, and is left as Exercise 19.4-1. ■

Corollary 19.5

On the simple path from any node going up toward a root, node ranks strictly increase. ■

Lemma 19.6

Every node has rank at most $n - 1$.

Proof Each node's rank starts at 0, and it increases only upon LINK operations. Because there are at most $n - 1$ UNION operations, there are also at most $n - 1$ LINK operations. Because each LINK operation either leaves all ranks alone or increases some node's rank by 1, all ranks are at most $n - 1$. ■

Lemma 19.6 provides a weak bound on ranks. In fact, every node has rank at most $\lceil \lg n \rceil$ (see Exercise 19.4-2). The looser bound of Lemma 19.6 suffices for our purposes, however.

Proving the time bound

In order to prove the $O(m \alpha(n))$ time bound, we'll use the potential method of amortized analysis from Section 16.3. In performing the amortized analysis, it will be convenient to assume that we invoke the LINK operation rather than the UNION operation. That is, since the parameters of the LINK procedure are pointers to two roots, we act as though we perform the appropriate FIND-SET operations separately. The following lemma shows that even if we count the extra FIND-SET operations induced by UNION calls, the asymptotic running time remains unchanged.

Lemma 19.7

Suppose that we convert a sequence S' of m' MAKE-SET, UNION, and FIND-SET operations into a sequence S of m MAKE-SET, LINK, and FIND-SET operations by turning each UNION into two FIND-SET operations followed by one LINK. Then, if sequence S runs in $O(m \alpha(n))$ time, sequence S' runs in $O(m' \alpha(n))$ time.

Proof Since each UNION operation in sequence S' is converted into three operations in S , we have $m' \leq m \leq 3m'$, so that $m = \Theta(m')$. Thus, an $O(m \alpha(n))$

time bound for the converted sequence S implies an $O(m' \alpha(n))$ time bound for the original sequence S' . ■

From now on, we assume that the initial sequence of m' MAKE-SET, UNION, and FIND-SET operations has been converted to a sequence of m MAKE-SET, LINK, and FIND-SET operations. We now prove an $O(m \alpha(n))$ time bound for the converted sequence and appeal to Lemma 19.7 to prove the $O(m' \alpha(n))$ running time of the original sequence of m' operations.

Potential function

The potential function we use assigns a potential $\phi_q(x)$ to each node x in the disjoint-set forest after q operations. For the potential Φ_q of the entire forest after q operations, sum the individual node potentials: $\Phi_q = \sum_x \phi_q(x)$. Because the forest is empty before the first operation, the sum is taken over an empty set, and so $\Phi_0 = 0$. No potential Φ_q is ever negative.

The value of $\phi_q(x)$ depends on whether x is a tree root after the q th operation. If it is, or if $x.rank = 0$, then $\phi_q(x) = \alpha(n) \cdot x.rank$.

Now suppose that after the q th operation, x is not a root and that $x.rank \geq 1$. We need to define two auxiliary functions on x before we can define $\phi_q(x)$. First we define

$$\text{level}(x) = \max \{k : x.p.rank \geq A_k(x.rank)\} . \quad (19.3)$$

That is, $\text{level}(x)$ is the greatest level k for which A_k , applied to x 's rank, is no greater than x 's parent's rank.

We claim that

$$0 \leq \text{level}(x) < \alpha(n) , \quad (19.4)$$

which we see as follows. We have

$$\begin{aligned} x.p.rank &\geq x.rank + 1 \quad (\text{by Lemma 19.4 because } x \text{ is not a root}) \\ &= A_0(x.rank) \quad (\text{by the definition (19.1) of } A_0(j)) , \end{aligned}$$

which implies that $\text{level}(x) \geq 0$, and

$$\begin{aligned} A_{\alpha(n)}(x.rank) &\geq A_{\alpha(n)}(1) \quad (\text{because } A_k(j) \text{ is strictly increasing}) \\ &\geq n \quad (\text{by the definition (19.2) of } \alpha(n)) \\ &> x.p.rank \quad (\text{by Lemma 19.6}) , \end{aligned}$$

which implies that $\text{level}(x) < \alpha(n)$.

For a given nonroot node x , the value of $\text{level}(x)$ monotonically increases over time. Why? Because x is not a root, its rank does not change. The rank of $x.p$

monotonically increases over time, since if $x.p$ is not a root then its rank does not change, and if $x.p$ is a root then its rank can never decrease. Thus, the difference between $x.rank$ and $x.p.rank$ monotonically increases over time. Therefore, the value of k needed for $A_k(x.rank)$ to overtake $x.p.rank$ monotonically increases over time as well.

The second auxiliary function applies when $x.rank \geq 1$:

$$\text{iter}(x) = \max \{i : x.p.rank \geq A_{\text{level}(x)}^{(i)}(x.rank)\} . \quad (19.5)$$

That is, $\text{iter}(x)$ is the largest number of times we can iteratively apply $A_{\text{level}(x)}$, applied initially to x 's rank, before exceeding x 's parent's rank.

We claim that when $x.rank \geq 1$, we have

$$1 \leq \text{iter}(x) \leq x.rank , \quad (19.6)$$

which we see as follows. We have

$$\begin{aligned} x.p.rank &\geq A_{\text{level}(x)}(x.rank) \quad (\text{by the definition (19.3) of level}(x)) \\ &= A_{\text{level}(x)}^{(1)}(x.rank) \quad (\text{by the definition (3.30) of functional iteration}) , \end{aligned}$$

which implies that $\text{iter}(x) \geq 1$. We also have

$$\begin{aligned} A_{\text{level}(x)}^{(x.rank+1)}(x.rank) &= A_{\text{level}(x)+1}(x.rank) \quad (\text{by the definition (19.1) of } A_k(j)) \\ &> x.p.rank \quad (\text{by the definition (19.3) of level}(x)) , \end{aligned}$$

which implies that $\text{iter}(x) \leq x.rank$. Note that because $x.p.rank$ monotonically increases over time, in order for $\text{iter}(x)$ to decrease, $\text{level}(x)$ must increase. As long as $\text{level}(x)$ remains unchanged, $\text{iter}(x)$ must either increase or remain unchanged.

With these auxiliary functions in place, we are ready to define the potential of node x after q operations:

$$\phi_q(x) = \begin{cases} \alpha(n) \cdot x.rank & \text{if } x \text{ is a root or } x.rank = 0 , \\ (\alpha(n) - \text{level}(x)) \cdot x.rank - \text{iter}(x) & \text{if } x \text{ is not a root and } x.rank \geq 1 . \end{cases} \quad (19.7)$$

We next investigate some useful properties of node potentials.

Lemma 19.8

For every node x , and for all operation counts q , we have

$$0 \leq \phi_q(x) \leq \alpha(n) \cdot x.rank .$$

Proof If x is a root or $x.rank = 0$, then $\phi_q(x) = \alpha(n) \cdot x.rank$ by definition. Now suppose that x is not a root and that $x.rank \geq 1$. We can obtain a lower bound on $\phi_q(x)$ by maximizing $\text{level}(x)$ and $\text{iter}(x)$. The bounds (19.4) and (19.6) give $\alpha(n) - \text{level}(x) \geq 1$ and $\text{iter}(x) \leq x.rank$. Thus, we have

$$\begin{aligned}
\phi_q(x) &= (\alpha(n) - \text{level}(x)) \cdot x.\text{rank} - \text{iter}(x) \\
&\geq x.\text{rank} - x.\text{rank} \\
&= 0.
\end{aligned}$$

Similarly, minimizing $\text{level}(x)$ and $\text{iter}(x)$ provides an upper bound on $\phi_q(x)$. By the bound (19.4), $\text{level}(x) \geq 0$, and by the bound (19.6), $\text{iter}(x) \geq 1$. Thus, we have

$$\begin{aligned}
\phi_q(x) &\leq (\alpha(n) - 0) \cdot x.\text{rank} - 1 \\
&= \alpha(n) \cdot x.\text{rank} - 1 \\
&< \alpha(n) \cdot x.\text{rank}.
\end{aligned}$$

■

Corollary 19.9

If node x is not a root and $x.\text{rank} > 0$, then $\phi_q(x) < \alpha(n) \cdot x.\text{rank}$.

■

Potential changes and amortized costs of operations

We are now ready to examine how the disjoint-set operations affect node potentials. Once we understand how each operation can change the potential, we can determine the amortized costs.

Lemma 19.10

Let x be a node that is not a root, and suppose that the q th operation is either a LINK or a FIND-SET. Then after the q th operation, $\phi_q(x) \leq \phi_{q-1}(x)$. Moreover, if $x.\text{rank} \geq 1$ and either $\text{level}(x)$ or $\text{iter}(x)$ changes due to the q th operation, then $\phi_q(x) \leq \phi_{q-1}(x) - 1$. That is, x 's potential cannot increase, and if it has positive rank and either $\text{level}(x)$ or $\text{iter}(x)$ changes, then x 's potential drops by at least 1.

Proof Because x is not a root, the q th operation does not change $x.\text{rank}$, and because n does not change after the initial n MAKE-SET operations, $\alpha(n)$ remains unchanged as well. Hence, these components of the formula for x 's potential remain the same after the q th operation. If $x.\text{rank} = 0$, then $\phi_q(x) = \phi_{q-1}(x) = 0$.

Now assume that $x.\text{rank} \geq 1$. Recall that $\text{level}(x)$ monotonically increases over time. If the q th operation leaves $\text{level}(x)$ unchanged, then $\text{iter}(x)$ either increases or remains unchanged. If both $\text{level}(x)$ and $\text{iter}(x)$ are unchanged, then $\phi_q(x) = \phi_{q-1}(x)$. If $\text{level}(x)$ is unchanged and $\text{iter}(x)$ increases, then it increases by at least 1, and so $\phi_q(x) \leq \phi_{q-1}(x) - 1$.

Finally, if the q th operation increases $\text{level}(x)$, it increases by at least 1, so that the value of the term $(\alpha(n) - \text{level}(x)) \cdot x.\text{rank}$ drops by at least $x.\text{rank}$. Because $\text{level}(x)$ increased, the value of $\text{iter}(x)$ might drop, but according to the bound (19.6), the drop is by at most $x.\text{rank} - 1$. Thus, the increase in poten-

tial due to the change in $\text{iter}(x)$ is less than the decrease in potential due to the change in $\text{level}(x)$, yielding $\phi_q(x) \leq \phi_{q-1}(x) - 1$. ■

Our final three lemmas show that the amortized cost of each MAKE-SET, LINK, and FIND-SET operation is $O(\alpha(n))$. Recall from equation (16.2) on page 456 that the amortized cost of each operation is its actual cost plus the change in potential due to the operation.

Lemma 19.11

The amortized cost of each MAKE-SET operation is $O(1)$.

Proof Suppose that the q th operation is MAKE-SET(x). This operation creates node x with rank 0, so that $\phi_q(x) = 0$. No other ranks or potentials change, and so $\Phi_q = \Phi_{q-1}$. Noting that the actual cost of the MAKE-SET operation is $O(1)$ completes the proof. ■

Lemma 19.12

The amortized cost of each LINK operation is $O(\alpha(n))$.

Proof Suppose that the q th operation is LINK(x, y). The actual cost of the LINK operation is $O(1)$. Without loss of generality, suppose that the LINK makes y the parent of x .

To determine the change in potential due to the LINK, note that the only nodes whose potentials may change are x, y , and the children of y just prior to the operation. We'll show that the only node whose potential can increase due to the LINK is y , and that its increase is at most $\alpha(n)$:

- By Lemma 19.10, any node that is y 's child just before the LINK cannot have its potential increase due to the LINK.
- From the definition (19.7) of $\phi_q(x)$, note that, since x was a root just before the q th operation, $\phi_{q-1}(x) = \alpha(n) \cdot x.\text{rank}$ at that time. If $x.\text{rank} = 0$, then $\phi_q(x) = \phi_{q-1}(x) = 0$. Otherwise,

$$\begin{aligned} \phi_q(x) &< \alpha(n) \cdot x.\text{rank} \quad (\text{by Corollary 19.9}) \\ &= \phi_{q-1}(x), \end{aligned}$$

and so x 's potential decreases.

- Because y is a root prior to the LINK, $\phi_{q-1}(y) = \alpha(n) \cdot y.\text{rank}$. After the LINK operation, y remains a root, so that y 's potential still equals $\alpha(n)$ times its rank after the operation. The LINK operation either leaves y 's rank alone or increases y 's rank by 1. Therefore, either $\phi_q(y) = \phi_{q-1}(y)$ or $\phi_q(y) = \phi_{q-1}(y) + \alpha(n)$.